

Parte 1

Importante: Los ejercicios de esta primera parte tienen como objetivo codificar las diferentes funciones básicas necesarias para la implementar un Trie.

A partir de estructuras definidas como :

```
class Trie:
    root = None

class TrieNode:
    parent = None
    children = None
    key = None
    isEndOfWord = False
```

Sugerencia 1: Para manejar múltiples nodos, el campo children puede contener una estructura **LinkedList** conteniendo **TrieNode**

Ejercicio 1

Crear un módulo de nombre **trie.py** que **implemente** las siguientes especificaciones de las operaciones elementales para el **TAD Trie** .

insert(T,element)

Descripción: insert un elemento en T, siendo T un Trie.

Entrada: El Trie sobre la cual se quiere agregar el elemento (Trie) y el valor del elemento (palabra) a agregar.

Salida: No hay salida definida

search(T,element)

Descripción: Verifica que un elemento se encuentre dentro del Trie

Entrada: El Trie sobre la cual se quiere buscar el elemento (Trie) y el valor del elemento (palabra)

Salida: Devuelve **False** o **True** según se encuentre el elemento.

```
def insert(T:Trie, element):
    #Crea la raíz si el Trie está vacío
    if T.root is None:
        T.root=TrieNode()

    current=T.root

    #Recorre cada letra de la palabra
```

```
for i in element:
    node=findKey(current, i)

    #Si la letra no existe, crea el nodo
    if node is None:
        node=TrieNode()
        node.key=i
        node.parent=current

        #Agrega el nodo a los hijos
        if current.children is None:
            current.children=[node]
        else:
            current.children.append(node)

    #Avanza al nodo de la letra
    current=node

current.isEndOfWord=True

def findKey(node:TrieNode, key):
    #devuelve el nodo con la key buscada dentro de children
    if node is None or node.children is None:
        return None

    for child in node.children:
        if child.key == key:
            return child
    return None

def search(T:Trie, element):
    #Recorre el Trie siguiendo cada letra
    current=T.root

    for i in element:
        node=findKey(current, i)

        #La palabra no existe
        if node is None:
            return False
```

```
current=node

#Verifica que sea una palabra completa
return current.isEndOfWord
```

Ejercicio 2 (no code)

Sabiendo que el orden de complejidad para el peor caso de la operación `search()` es de $O(m \cdot |\Sigma|)$.
Proponga una versión de la operación `search()` cuya complejidad sea $O(m)$.

Como se trabaja con un alfabeto fijo, se pueden usar estructuras de tamaño estático como un array, cuyo tiempo de acceso es lineal. Si le otorgáramos a cada letra un lugar determinado dentro de cada array, por ejemplo si “a” siempre está en `children[0]`, “b” en `children[1]`, etc, entonces por cada letra podríamos verificar su existencia directamente accediendo a dicha posición determinada (en $O(1)$). Lo que la complejidad de la búsqueda quede definida por el largo de la palabra.

El problema con esta implementación sería el espacio, ya que si el árbol no está muy lleno tendríamos muchos espacios de memoria ocupados de forma innecesaria.

Ejercicio 3

`delete(T,element)`

Descripción: Elimina un elemento se encuentre dentro del Trie

Entrada: El Trie sobre la cual se quiere eliminar el elemento (Trie) y el valor del elemento (palabra) a eliminar.

Salida: Devuelve `False` o `True` según se haya eliminado el elemento.

```
def delete(T:Trie, element):
    #Recorre el Trie buscando la palabra
    current=T.root

    for i in element:
        node=findKey(current, i)

        #La palabra no existe
        if node is None:
            return False

        #Si el nodo no tiene hijos, se puede eliminar
        if not node.isEndOfWord and (node.children is None or len(node.children)==0):

            #Busca y elimina el nodo de su padre
```

```
        for j in range(len(node.parent.children)):
            if node.parent.children[j] == node:
                node.parent.children.pop(j)
                break

        current=node

        #No era una palabra completa
        if not current.isEndOfWord:
            return False

        #Desmarca el final de la palabra
        current.isEndOfWord=False

    return True
```

Parte 2

Ejercicio 4

Implementar un algoritmo que dado un árbol **Trie T**, un patrón **p (prefijo)** y un entero **n**, escriba todas las palabras del árbol que empiezan por **p** y sean de longitud **n**.

```
def prefijoLongitud(T:Trie, p, n:int)->list:

    result=[]

    #El prefijo no puede superar la longitud buscada
    if len(p) > n:
        return result

    current=T.root

    #Busca el prefijo en el Trie
    for i in p:
        node=findKey(current, i)

        #El prefijo no existe
        if node is None:
            return result
```

```
        current=node

    #Busca palabras hasta alcanzar longitud n
    prefijoLongitudR(current, p, n - len(p), result)

    return result

def prefijoLongitudR(node:TrieNode, subcadena, resto:int, result:list):

    #Si llegó a longitud n, verifica si es palabra
    if resto == 0:
        if node.isEndOfWord:
            result.append(subcadena)
        return

    #No hay más letras para continuar
    if node.children is None:
        return

    #Explora todos los hijos posibles
    for child in node.children:
        prefijoLongitudR(child, subcadena + child.key, resto - 1, result)
```

Ejercicio 5

Implementar un algoritmo que dado los **Trie** T1 y T2 devuelva **True** si estos pertenecen al mismo documento y **False** en caso contrario. Se considera que un **Trie** pertenece al mismo documento cuando:

1. Ambos Trie sean iguales (esto se debe cumplir)
2. ~~El Trie T1 contiene un subconjunto de las palabras del Trie T2~~
3. Si la implementación está basada en LinkedList, considerar el caso donde las palabras hayan sido insertadas en un orden diferente.

En otras palabras, analizar si todas las palabras de T1 se encuentran en T2.

Analizar el costo computacional.

Para encontrar todas las palabras de ambos trie se deben recorrer todos sus nodos. Sea M la suma de los nodos de T1 Y T2.

Luego, en el caso de que sí sean el mismo documento, se recorren las dos listas de palabras de forma anidada, sea N la cantidad de palabras en ambos tries.
Entonces la complejidad está dada por $O(M+N^2)$

```
def mismoDoc(T1:Trie, T2:Trie)->bool:

    #Obtiene las palabras de ambos Tries
    words1=getPalabras(T1)
    words2=getPalabras(T2)

    #Si tienen distinta cantidad, no son iguales
    if len(words1) != len(words2):
        return False

    #Verifica que todas las palabras coincidan
    for w1 in words1:
        if not w1 in words2:
            return False

    return True

def getPalabras(T):
    #Obtiene todas las palabras del Trie
    result=[]
    palabrasR(T.root, "", result)
    return result

def palabrasR(node, subcadena, result):

    if node is None:
        return

    #Guarda la palabra si llegó a su final
    if node.isEndOfWord:
        result.append(subcadena)

    if node.children is None:
        return

    #Recorre todos los hijos
    for child in node.children:
        palabrasR(child, subcadena + child.key, result)
```

Ejercicio 6

Implemente un algoritmo que dado el **Trie** T devuelva **True** si existen en el documento T dos cadenas invertidas. Dos cadenas son invertidas si se leen de izquierda a derecha y contiene los mismos caracteres que si se lee de derecha a izquierda, ej: **abcd** y **dcba** son cadenas invertidas, **gfdsa** y **asdfg** son cadenas invertidas, sin embargo **abcd** y **dcka** no son invertidas ya que difieren en un carácter.

```
def palindromo(T):  
    #Obtiene todas las palabras del trie  
    words=getPalabras(T)  
    for w in words:  
        #Invierte cada palabra y la busca. Si encuentra alguna coincidencia, devuelve  
        True  
        r=w[::-1]  
        if r in words:  
            return True  
    return False
```

Ejercicio 7

Un corrector ortográfico interactivo utiliza un **Trie** para representar las palabras de su diccionario. Queremos añadir una función de auto-completar (al estilo de la tecla TAB en Linux): cuando estamos a medio escribir una palabra, si sólo existe una forma correcta de continuarla entonces debemos indicarlo.

Implementar la función **autoCompletar(Trie, cadena)** dentro del módulo **trie.py**, que dado el árbol **Trie T** y la cadena devuelve la forma de auto-completar la palabra. Por ejemplo, para la llamada **autoCompletar(T, 'groen')** devolvería **"land"**, ya que podemos tener **"groenlandia"** o **"groenlandés"** (en este ejemplo la palabra groenlandia y groenlandés pertenecen al documento que representa el Trie). Si hay varias formas o ninguna, devolvería la cadena vacía. Por ejemplo, **autoCompletar(T, ma)** devolvería **""** (cadena vacía) si **T** presenta las cadenas **"madera"** y **"mama"**.

```
def autoCompletar(T, cadena):  
    current=T.root  
    #Busca la cadena introducida y llega hasta su última letra
```

```
for i in cadena:

    node=findKey(current, i)

    if node is None:

        return ""

    current=node

subcadena=""

while current is not None:

    #Avanza mientras haya un único camino posible

    if current.children is not None:

        lenChildren=len(current.children)

    else:

        lenChildren=0

    if current.isEndOfWord and lenChildren > 0:

        #Palabra completa con más caminos: se devuelve lo que se tiene para evitar
ambigüedad

        return subcadena

    if current.isEndOfWord and lenChildren == 0:

        #Única hoja alcanzada: completado único encontrado

        return subcadena

    if lenChildren != 1:
```



```
#más de 1 hijo: ambigüedad, se devuelve lo obtenido hasta ahí  
  
return subcadena  
  
#Queda un solo camino a recorrer porque len(children) es 1  
  
current=current.children[0]  
  
subcadena += current.key
```